

Render Farm - Cheat Sheet

Real Redis + real BullMQ. Workers are separate OS processes. A crash is a real SIGKILL, recovery is real stalled-job detection.

One-line mental model: Frames enter a shared queue; idle workers pull them; the orchestrator merges BullMQ lifecycle events and worker telemetry into one WorldState and broadcasts it to the browser at 5 Hz. An autonomous Director seeds each cycle and injects drama (crash, scale up, scale down) every 5 to 10 seconds, forever.

1 Run it & drive it

- **Start everything** with `npm install`, then `npm run redis:start` (or `docker compose up -d`) to bring up Redis, then `npm run dev` to launch the orchestrator (which forks the workers) alongside the Vite SPA. Open `http://localhost:5173` and it cycles on its own.
- **Pause / Resume** freezes and resumes only the Director's autonomous drama; frames already in flight keep processing while paused.
- **+ Inject 5** adds five more frames to the current cycle through the same seed path the Director uses.
- **Kill a node** sends a real `SIGKILL` to a busy worker so you can watch fault recovery; it works even while paused and falls back to any node if none are busy.
- **Reset** obliterates the queue, clears the board, bumps the cycle number, and restarts from seeding.

2 The four behaviors, briefly

- **Load balancing** is emergent from a single shared queue: whichever worker is idle pulls the next frame, with no assignment logic anywhere in the code.
- **Fault tolerance** comes from BullMQ stalled-job detection: a killed worker's Redis lock expires, a survivor notices the orphan on its next sweep, and the frame re-queues and completes elsewhere.
- **Autoscaling** is the Director reacting to queue depth: a deep backlog forks a worker, a draining queue retires an idle one, always within the min/max bounds.
- **Priorities** use BullMQ's own priority field so a flagged frame overtakes normal frames still waiting, rather than the front-end merely re-sorting cards.

3 Frame & stage model

- **A job is one render frame**, identified `f<cycle>-<seq>` (for example `f1-3`), and that id is used directly as the BullMQ `jobId` so lifecycle events map straight onto frames.
- **Two timed stages run in strict order**, RENDERING then COMPOSITING, each roughly `STAGE_MS` (2500 ms), so a frame takes about five seconds end to end and its motion is continuously visible.

QUEUED → RENDERING → COMPOSITING → DONE

4 Two channels of truth

- **BullMQ QueueEvents is authoritative for lifecycle** - added, completed, stalled, failed - and answers "where is this frame"; the orchestrator folds it with `applyQueueEvent`.
- **The `worker:telemetry` pub/sub channel carries node detail** - which node owns which frame, current stage, live percent, heartbeat, per-node completed count - and is folded with `applyTelemetry`.
- **The two streams are unordered relative to each other**, so the merge is written to tolerate stragglers: a stalled event unconditionally clears frame ownership, and telemetry only syncs a frame that still reports a stage.
- **Both fold into one immutable WorldStore**, and a separate broadcaster ships the whole snapshot at 5 Hz, decoupling noisy inputs from the browser send rate.

5 The Director cycle engine

- **The cycle is a four-phase machine** - SEED, RUNNING, COMPLETE, RESET - advanced by a timer firing at a random 5 to 10 second interval, looping forever.
- **All decisions live in a pure reducer**, `reduceDirector(state, action, ctx)`, which returns the next state plus a list of effects and performs no side effects, which is what makes the whole engine unit-testable without Redis.
- **The impure driver** `runDirector` **applies effects**: seed enqueues frames, spawn forks a worker, kill retires an idle node, crash SIGKILLS a busy one, resetQueue obliterates and clears.
- **Randomness is injected from outside** the reducer (two rolls in the context), so tests can force an exact crash or an exact no-crash deterministically.
- **Manual commands reuse the same paths**, so there is one tested code path per action rather than a parallel manual implementation.

6 Crash recovery timing

- **SIGKILL leaves a lock behind**: the worker dies mid-frame holding its Redis lock, which is never renewed again because the process is gone.
- **The lock expires after** `LOCK_DURATION_MS` (4000 ms), after which the job is eligible to be seen as stalled by any surviving worker.
- **Survivors sweep every** `STALLED_INTERVAL_MS` (2000 ms); the next sweep after expiry re-queues the orphan back to QUEUED, and an idle worker then completes it.
- **Worst-case recovery =** `LOCK_DURATION_MS` + `STALLED_INTERVAL_MS` = 6000 ms, landing recovery in a 4 to 8 second window - BullMQ's 30 s defaults would be far too slow to watch.
- **Retry headroom is generous** (`JOB_ATTEMPTS` 20, `MAX_STALLED_COUNT` 10) so a frame crashed several times still succeeds instead of failing.

7 Autoscaling & priorities

- **Queue depth = waiting + prioritized jobs**, the single number the Director scales on.
- **Scale up** forks one worker when depth strictly exceeds `SCALE_UP_DEPTH` (6) and the pool is below `MAX_NODES` (6).
- **Scale down** retires one *idle* worker when depth strictly drops below `SCALE_DOWN_DEPTH` (2) and the pool is above `MIN_NODES` (2), so in-flight work is never interrupted.
- **Comparisons are strict and one action per tick**, pinned by boundary unit tests, so a depth sitting on a threshold does not oscillate the pool.
- **Priority frames use BullMQ priority 1 vs 5** (lower is higher); about `HIGH_PRIORITY_RATIO` (0.15) of a batch is flagged, and they overtake the *waiting* line, not frames already rendering.

8 Worker process facts

- **Each worker is a forked Node process** launched via `fork()` under `tsx`, with its identity and timing knobs passed as environment variables (`NODE_ID`, `STAGE_MS`, lock and stalled intervals, and so on).
- **Each worker opens two Redis connections** - one for the BullMQ worker (`maxRetriesPerRequest: null`) and one dedicated telemetry publisher - and runs at concurrency 1, so one node processes one frame at a time.
- **An idle worker heartbeats** every `HEARTBEAT_INTERVAL_MS` (2000 ms) so it does not vanish from the strip between frames; while busy, progress ticks already prove it is alive.
- **Graceful retire is** `SIGTERM` (clean close, removed at once); **a crash is** `SIGKILL` (marked red, lingers 1500 ms on screen, then removed). Node ids come from a monotonic counter and are never reused.

9 Wire contract (@demo/shared)

- **Constants everyone shares**: `queue` `render-frames`, `channel` `worker:telemetry`, the four stages, event levels `info/success/warn/danger`, and command types.
- **Key shapes**: `Frame` (id, cycle, stage, nodeId, pct, priority, failed), `WorkerNode` (id, pid, state, frameId, pct, completed), `TelemetryMsg`, and the broadcast `WorldState`.

NodeState	Source
idle / rendering / compositing / spawning	worker telemetry
crashed	orchestrator on unexpected exit

10 Tunables (server/src/config)

Env var	Default
BATCH_SIZE	16
HIGH_PRIORITY_RATIO	0.15
TICK_MIN_MS / MAX_MS	5000 / 10000
STAGE_MS	2500
MIN_NODES / MAX_NODES	2 / 6
SCALE_UP / DOWN_DEPTH	6 / 2
LOCK_DURATION_MS	4000
STALLED_INTERVAL_MS	2000
HEARTBEAT_INTERVAL_MS	2000
JOB_ATTEMPTS / MAX_STALLED	20 / 10
BROADCAST_HZ	5

11 Testing & gotchas

- **Stop `npm run dev` before server integration tests** - the demo and the tests share one Redis queue, and the tests flush Redis, so a running demo steals test jobs and the tests wipe the demo.
- **Integration tests run sequentially** because they all share that single queue; they need a real Redis on 6379.
- **Unit tests need no Redis:** the Director reducer, the world-state folds, the worker's stage ordering, and the command router are all pure or mockable.
- **To see a permanent FAILED card**, force the retry budget down with `MAX_STALLED_COUNT=0 JOB_ATTEMPTS=1` and kill a node; the frame still lands in DONE (red-bordered) so a cycle can never deadlock.

12 Where things live

Concern	File
Bootstrap / wiring	server/src/index.ts
Decision reducer	services/director/reduceDirector.ts
Effect driver	services/director/runDirector.ts
Fork / kill workers	services/nodePool/createNodePool.ts
Lifecycle fold	worldState/applyQueueEvent.ts
Telemetry fold	worldState/applyTelemetry.ts
Snapshot broadcast	websocket/createBroadcaster.ts
Worker entry	worker/src/index.ts
Stage pipeline	worker/src/processFrame.ts
Browser socket hook	web/src/state/useOrchestrator.ts
Shared contract	shared/src/{types,constants}.ts